Python Dictionary

The Complete Guide - Every Concept, Every Method

Simple explanations | Easy examples | Real outputs | Practice questions

No.	Chapter
1	What is a Dictionary?
2	Creating a Dictionary (6 ways)
3	Rules for Keys and Values
4	Accessing Values
5	Adding and Changing Items
6	Removing Items
7	All Dictionary Methods (with examples)
8	Looping Through a Dictionary
9	Useful Built-in Functions with Dictionaries
10	Merging Dictionaries
11	Nested Dictionaries
12	Dictionary Comprehension
13	Sorting a Dictionary
14	Copying: Shallow vs Deep
15	Dictionary Views are Live
16	defaultdict and Counter (Bonus)
17	Real-Life Mini Programs
18	Common Mistakes
19	Quick Reference Table
20	Practice Questions with Answers

1. What is a Dictionary?

A **dictionary** stores data as **key : value** pairs. Think of a real dictionary or a phone contact list: you look up a **word (key)** and get its **meaning (value)**. You do not need to know the position of an item, only its key.

Example

```
student = {"name": "Riya", "age": 19, "branch": "CSE"}
print(student)
print(type(student))
print(student["name"])
```

Output

```
{'name': 'Riya', 'age': 19, 'branch': 'CSE'}
<class 'dict'>
Riya
```

Key facts at a glance

- Written inside curly braces **{ }**, each pair as **key: value**, separated by commas.
- **Mutable**: you can add, change and remove items after creating it.
- **Ordered**: items keep the order in which you inserted them (guaranteed from Python 3.7).
- **Keys are unique**: the same key cannot appear twice.
- Fast lookup: finding a value by key is very quick, even for huge dictionaries.

Example

```
d = {"a": 1, "b": 2}
print(len(d))        # number of key-value pairs
print(bool({}))      # empty dictionary is False
print(bool(d))       # non-empty dictionary is True
```

Output

```
2
False
True
```

2. Creating a Dictionary (6 ways)

Way 1: Curly braces (most common)

Example

```
person = {"name": "Aman", "city": "Delhi"}
print(person)
```

Output

```
{'name': 'Aman', 'city': 'Delhi'}
```

Way 2: Empty dictionary, then fill it

Example

```
d = {}
print(d, type(d))
d["x"] = 10
print(d)
```

Output

```
{} <class 'dict'>
{'x': 10}
```

Note: `{}` makes an empty **dictionary**, not a set. For an empty set use `set()`.

Way 3: dict() with keyword arguments

Example

```
d = dict(name="Aman", age=20)
print(d)
```

Output

```
{'name': 'Aman', 'age': 20}
```

Note: Here keys must be simple names (no spaces, no quotes) and they become strings automatically.

Way 4: dict() from a list of pairs (tuples)

Example

```
pairs = [("a", 1), ("b", 2), ("c", 3)]
d = dict(pairs)
print(d)
```

Output

```
{'a': 1, 'b': 2, 'c': 3}
```

Way 5: dict() with zip() - two separate lists

Example

```
keys = ["name", "age", "city"]
values = ["Neha", 21, "Jaipur"]
d = dict(zip(keys, values))
print(d)
```

Output

```
{'name': 'Neha', 'age': 21, 'city': 'Jaipur'}
```

Way 6: dict.fromkeys() - same value for many keys

Example

```
subjects = ["maths", "physics", "chem"]
marks = dict.fromkeys(subjects, 0)
print(marks)
print(dict.fromkeys(["a", "b"]))   # value defaults to None
```

Output

```
{'maths': 0, 'physics': 0, 'chem': 0}
{'a': None, 'b': None}
```

3. Rules for Keys and Values

Rule 1: Keys must be unique

If you repeat a key, the **last value wins** and the earlier one is overwritten.

Example

```
d = {"a": 1, "b": 2, "a": 99}
print(d)
```

Output

```
{'a': 99, 'b': 2}
```

Rule 2: Keys must be immutable (cannot change)

Allowed keys: **string, number, tuple** (tuple only if it contains immutable items), boolean. Not allowed: **list, set, dictionary**.

Example

```
d = {}
d["name"] = 1          # string key
d[10] = "ten"          # int key
d[(1, 2)] = "point"    # tuple key
print(d)
try:
    d[[1, 2]] = "bad"  # list key
except TypeError as e:
    print("Error:", e)
```

Output

```
{'name': 1, 10: 'ten', (1, 2): 'point'}
Error: unhashable type: 'list'
```

Rule 3: Values can be anything, and can repeat

Example

```
d = {"a": 5, "b": 5, "c": [1, 2, 3], "d": {"x": 1}, "e": None, "f": 3.14}
print(d)
```

Output

```
{'a': 5, 'b': 5, 'c': [1, 2, 3], 'd': {'x': 1}, 'e': None, 'f': 3.14}
```

Rule 4: 1 and True (and 1.0) are the same key

Example

```
d = {1: "one", True: "yes", 1.0: "float"}
print(d)
```

Output

```
{1: 'float'}
```

Python treats `1`, `True` and `1.0` as equal, so they collapse into one key.

4. Accessing Values

Using square brackets [ ]

Example

```
car = {"brand": "Tata", "model": "Nexon", "year": 2024}
print(car["brand"])
print(car["year"])
```

Output

```
Tata
2024
```

Missing key gives KeyError

Example

```
car = {"brand": "Tata"}
try:
    print(car["color"])
except KeyError as e:
    print("KeyError for key:", e)
```

Output

```
KeyError for key: 'color'
```

Safe way: get()

`get(key)` returns **None** if the key is missing. `get(key, default)` returns your chosen default. It never crashes.

Example

```
car = {"brand": "Tata", "year": 2024}
print(car.get("brand"))
print(car.get("color"))             # None
print(car.get("color", "unknown"))  # default value
```

Output

```
Tata
None
unknown
```

Checking if a key exists: in / not in

Example

```
car = {"brand": "Tata", "year": 2024}
print("brand" in car)
print("color" in car)
print("color" not in car)
print("Tata" in car)       # in checks KEYS, not values
print("Tata" in car.values())
```

Output

```
True
False
True
False
True
```

5. Adding and Changing Items

Add a new key / change an existing key

Same syntax for both: `dict[key] = value`. If the key exists it is updated, otherwise it is added.

Example

```
s = {"name": "Riya", "age": 19}
s["city"] = "Delhi"      # add new
s["age"] = 20            # change existing
print(s)
```

Output

```
{'name': 'Riya', 'age': 20, 'city': 'Delhi'}
```

Change many at once with update()

Example

```
s = {"name": "Riya", "age": 19}
s.update({"age": 21, "branch": "CSE"})
print(s)
s.update(city="Pune", cgpa=8.5)     # keyword style also works
print(s)
```

Output

```
{'name': 'Riya', 'age': 21, 'branch': 'CSE'}
{'name': 'Riya', 'age': 21, 'branch': 'CSE', 'city': 'Pune', 'cgpa': 8.5}
```

Add only if missing: setdefault()

Example

```
s = {"name": "Riya"}
s.setdefault("age", 18)       # key missing -> added
s.setdefault("name", "XXX")   # key exists -> unchanged
print(s)
```

Output

```
{'name': 'Riya', 'age': 18}
```

6. Removing Items

pop(key) - remove a key and return its value

Example

```
d = {"a": 1, "b": 2, "c": 3}
v = d.pop("b")
print("removed value:", v)
print(d)
print(d.pop("zzz", "not found"))   # default avoids KeyError
```

Output

```
removed value: 2
{'a': 1, 'c': 3}
not found
```

popitem() - remove the LAST inserted pair

Example

```
d = {"a": 1, "b": 2, "c": 3}
item = d.popitem()
print(item)
print(d)
```

Output

```
('c', 3)
{'a': 1, 'b': 2}
```

del - delete a key (or the whole dictionary)

Example

```
d = {"a": 1, "b": 2, "c": 3}
del d["a"]
print(d)
del d            # deletes the variable itself
try:
    print(d)
except NameError as e:
    print("Error:", e)
```

Output

```
{'b': 2, 'c': 3}
Error: name 'd' is not defined
```

clear() - empty the dictionary

Example

```
d = {"a": 1, "b": 2}
d.clear()
print(d)
```

Output

```
{}
```

Tool	What it does	If key missing
pop(key)	removes key, returns value	KeyError (unless default given)
popitem()	removes last pair, returns (key, value)	KeyError if dictionary is empty
del d[key]	removes key, returns nothing	KeyError
clear()	removes everything	works on empty too
del d	deletes the dictionary variable	NameError if used later

7. All Dictionary Methods (with examples)

Python dictionaries have exactly **11 methods**. Each one is explained below with a simple example.

#	Method	Purpose
1	keys()	all keys
2	values()	all values
3	items()	all (key, value) pairs
4	get()	safe access
5	update()	merge / change items
6	pop()	remove key, return value
7	popitem()	remove last pair
8	setdefault()	get key; add if missing
9	copy()	shallow copy
10	clear()	remove all items
11	fromkeys()	new dict from keys

7.1 keys()

Example

```
d = {"a": 1, "b": 2, "c": 3}
print(d.keys())
print(list(d.keys()))
```

Output

```
dict_keys(['a', 'b', 'c'])
['a', 'b', 'c']
```

7.2 values()

Example

```
d = {"a": 1, "b": 2, "c": 3}
print(d.values())
print(list(d.values()))
print(sum(d.values()))
```

Output

```
dict_values([1, 2, 3])
[1, 2, 3]
6
```

7.3 items()

Example

```
d = {"a": 1, "b": 2, "c": 3}
print(d.items())
print(list(d.items()))
```

Output

```
dict_items([('a', 1), ('b', 2), ('c', 3)])
[('a', 1), ('b', 2), ('c', 3)]
```

7.4 get(key, default)

Example

```
d = {"apple": 50, "mango": 80}
print(d.get("apple"))
print(d.get("kiwi", 0))
```

Output

```
50
0
```

7.5 update(other)

Example

```
d = {"a": 1, "b": 2}
d.update({"b": 20, "c": 30})
print(d)
d.update([("d", 4), ("e", 5)])      # list of pairs also works
print(d)
```

Output

```
{'a': 1, 'b': 20, 'c': 30}
{'a': 1, 'b': 20, 'c': 30, 'd': 4, 'e': 5}
```

7.6 pop(key, default)

Example

```
d = {"a": 1, "b": 2}
print(d.pop("a"))
print(d)
print(d.pop("x", -1))
```

Output

```
1
{'b': 2}
-1
```

7.7 popitem()

Example

```
d = {"a": 1, "b": 2, "c": 3}
print(d.popitem())
print(d.popitem())
print(d)
```

Output

```
('c', 3)
('b', 2)
{'a': 1}
```

7.8 setdefault(key, default)

Returns the value of the key. If the key does not exist, it adds the key with the default and returns the default.

Example

```
d = {"a": 1}
print(d.setdefault("a", 100))   # exists -> returns 1
print(d.setdefault("b", 200))   # missing -> adds, returns 200
print(d)
```

Output

```
1
200
{'a': 1, 'b': 200}
```

7.9 copy()

Example

```
d = {"a": 1, "b": 2}
c = d.copy()
c["a"] = 999
print("original:", d)
print("copy    :", c)
```

Output

```
original: {'a': 1, 'b': 2}
copy    : {'a': 999, 'b': 2}
```

7.10 clear()

Example

```
d = {"a": 1, "b": 2}
d.clear()
print(d, len(d))
```

Output

```
{} 0
```

7.11 fromkeys(keys, value)

Example

```
d = dict.fromkeys(["x", "y", "z"], 0)
print(d)
```

Output

```
{'x': 0, 'y': 0, 'z': 0}
```

Note: With `fromkeys`, all keys share the **same** value object. If the value is a list, every key points to the same list. See Chapter 18.

8. Looping Through a Dictionary

Loop over keys (default)

Example

```
marks = {"maths": 90, "physics": 85, "chem": 78}
for key in marks:
    print(key)
```

Output

```
maths
physics
chem
```

Loop over values

Example

```
marks = {"maths": 90, "physics": 85, "chem": 78}
for v in marks.values():
    print(v)
```

Output

```
90
85
78
```

Loop over keys and values together (best way)

Example

```
marks = {"maths": 90, "physics": 85, "chem": 78}
for subject, score in marks.items():
    print(subject, "->", score)
```

Output

```
maths -> 90
physics -> 85
chem -> 78
```

Loop with index using enumerate()

Example

```
marks = {"maths": 90, "physics": 85, "chem": 78}
for i, (subject, score) in enumerate(marks.items(), start=1):
    print(i, subject, score)
```

Output

```
1 maths 90
2 physics 85
3 chem 78
```

Note: Never add or delete keys while looping over a dictionary; it raises `RuntimeError`. Loop over `list(d)` (a copy) if you must change it.

Example

```
d = {"a": 1, "b": 2, "c": 3}
for k in list(d):          # safe: loops over a copy of the keys
    if d[k] == 2:
        del d[k]
print(d)
```

Output

```
{'a': 1, 'c': 3}
```

9. Useful Built-in Functions with Dictionaries

Example

```
d = {"a": 30, "b": 10, "c": 20}
print(len(d))                 # number of pairs
print(sorted(d))              # sorted keys
print(min(d), max(d))         # smallest / largest KEY
print(min(d.values()), max(d.values()))
print(sum(d.values()))
print(any(v > 25 for v in d.values()))
print(all(v > 5 for v in d.values()))
print(list(reversed(d)))      # reversed key order (Python 3.8+)
print(str(d))
```

Output

```
3
['a', 'b', 'c']
a c
10 30
60
True
True
['c', 'b', 'a']
{'a': 30, 'b': 10, 'c': 20}
```

Example

```
d = {"a": 30, "b": 10, "c": 20}
best = max(d, key=d.get)      # key that has the biggest value
print("highest:", best, d[best])
worst = min(d, key=d.get)
print("lowest :", worst, d[worst])
```

Output

```
highest: a 30
lowest : b 10
```

10. Merging Dictionaries

Method 1: update() (changes the first dictionary)

Example

```
a = {"x": 1, "y": 2}
b = {"y": 20, "z": 30}
a.update(b)
print(a)
```

Output

```
{'x': 1, 'y': 20, 'z': 30}
```

Method 2: the | operator (Python 3.9+, makes a new dictionary)

Example

```
a = {"x": 1, "y": 2}
b = {"y": 20, "z": 30}
c = a | b
print(c)
print(a)       # a is unchanged
a |= b         # in-place merge, same as update()
print(a)
```

Output

```
{'x': 1, 'y': 20, 'z': 30}
{'x': 1, 'y': 2}
{'x': 1, 'y': 20, 'z': 30}
```

Method 3: ** unpacking

Example

```
a = {"x": 1, "y": 2}
b = {"y": 20, "z": 30}
c = {**a, **b}
print(c)
```

Output

```
{'x': 1, 'y': 20, 'z': 30}
```

In all methods, when a key is in both dictionaries, the value from the **right-hand / later** dictionary wins.

11. Nested Dictionaries

A dictionary can hold other dictionaries as values. This is great for storing records.

Example

```
students = {
    "s1": {"name": "Riya", "marks": 88},
    "s2": {"name": "Aman", "marks": 92},
}
print(students["s1"])
print(students["s2"]["name"])
```

Output

```
{'name': 'Riya', 'marks': 88}
Aman
```

Add, change and delete inside a nested dictionary

Example

```
students = {"s1": {"name": "Riya", "marks": 88}}
students["s2"] = {"name": "Aman", "marks": 92}   # add a record
students["s1"]["marks"] = 95                     # change inner value
students["s1"]["city"] = "Delhi"                 # add inner key
del students["s2"]["marks"]                      # delete inner key
print(students)
```

Output

```
{'s1': {'name': 'Riya', 'marks': 95, 'city': 'Delhi'}, 's2': {'name': 'Aman'}}
```

Loop through a nested dictionary

Example

```
students = {
    "s1": {"name": "Riya", "marks": 88},
    "s2": {"name": "Aman", "marks": 92},
}
for sid, info in students.items():
    print("ID:", sid)
    for k, v in info.items():
        print("   ", k, "=", v)
```

Output

```
ID: s1
    name = Riya
    marks = 88
ID: s2
    name = Aman
    marks = 92
```

Safe access to deep values

Example

```
data = {"user": {"profile": {"city": "Delhi"}}}
print(data.get("user", {}).get("profile", {}).get("city"))
print(data.get("user", {}).get("account", {}).get("id", "N/A"))
```

Output

```
Delhi
N/A
```

12. Dictionary Comprehension

A short one-line way to build a dictionary. Pattern: `{key: value for item in iterable}`

Example

```
squares = {n: n * n for n in range(1, 6)}
print(squares)
```

Output

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

With a condition (filter)

Example

```
squares = {n: n * n for n in range(1, 11) if n % 2 == 0}
print(squares)
```

Output

```
{2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```

Change values of an existing dictionary

Example

```
prices = {"pen": 10, "book": 100, "bag": 500}
discounted = {k: v * 0.9 for k, v in prices.items()}
print(discounted)
```

Output

```
{'pen': 9.0, 'book': 90.0, 'bag': 450.0}
```

Filter an existing dictionary

Example

```
marks = {"A": 45, "B": 82, "C": 67, "D": 91}
passed = {k: v for k, v in marks.items() if v >= 50}
print(passed)
```

Output

```
{'B': 82, 'C': 67, 'D': 91}
```

Swap keys and values

Example

```
d = {"a": 1, "b": 2, "c": 3}
swapped = {v: k for k, v in d.items()}
print(swapped)
```

Output

```
{1: 'a', 2: 'b', 3: 'c'}
```

Build from two lists

Example

```
names = ["Riya", "Aman", "Neha"]
marks = [88, 92, 79]
d = {n: m for n, m in zip(names, marks)}
print(d)
```

Output

```
{'Riya': 88, 'Aman': 92, 'Neha': 79}
```

With if-else for the value

Example

```
marks = {"A": 45, "B": 82, "C": 67}
result = {k: ("Pass" if v >= 50 else "Fail") for k, v in marks.items()}
print(result)
```

Output

```
{'A': 'Fail', 'B': 'Pass', 'C': 'Pass'}
```

13. Sorting a Dictionary

Dictionaries keep insertion order, so to sort them you create a **new** sorted dictionary.

Sort by key

Example

```
d = {"banana": 3, "apple": 5, "cherry": 2}
print(dict(sorted(d.items())))
```

Output

```
{'apple': 5, 'banana': 3, 'cherry': 2}
```

Sort by value

Example

```
d = {"banana": 3, "apple": 5, "cherry": 2}
print(dict(sorted(d.items(), key=lambda item: item[1])))
```

Output

```
{'cherry': 2, 'banana': 3, 'apple': 5}
```

Sort by value, highest first

Example

```
d = {"banana": 3, "apple": 5, "cherry": 2}
print(dict(sorted(d.items(), key=lambda item: item[1], reverse=True)))
```

Output

```
{'apple': 5, 'banana': 3, 'cherry': 2}
```

Top 2 items

Example

```
d = {"banana": 3, "apple": 5, "cherry": 2, "kiwi": 9}
top2 = dict(sorted(d.items(), key=lambda i: i[1], reverse=True)[:2])
print(top2)
```

Output

```
{'kiwi': 9, 'apple': 5}
```

14. Copying: Shallow vs Deep

Wrong way: using = does NOT copy

Example

```
a = {"x": 1}
b = a            # b is just another name for the same dictionary
b["x"] = 100
print(a)
print(a is b)
```

Output

```
{'x': 100}
True
```

Shallow copy: copy() or dict()

Example

```
a = {"x": 1, "y": 2}
b = a.copy()
c = dict(a)
b["x"] = 100
print(a, b, c)
```

Output

```
{'x': 1, 'y': 2} {'x': 100, 'y': 2} {'x': 1, 'y': 2}
```

Problem: shallow copy shares inner objects

Example

```
a = {"name": "Riya", "marks": [80, 90]}
b = a.copy()
b["marks"].append(100)     # changes the SAME inner list
print(a)
print(b)
```

Output

```
{'name': 'Riya', 'marks': [80, 90, 100]}
{'name': 'Riya', 'marks': [80, 90, 100]}
```

Deep copy: fully independent

Example

```
import copy
a = {"name": "Riya", "marks": [80, 90]}
b = copy.deepcopy(a)
b["marks"].append(100)
print(a)
print(b)
```

Output

```
{'name': 'Riya', 'marks': [80, 90]}
{'name': 'Riya', 'marks': [80, 90, 100]}
```

15. Dictionary Views are Live

`keys()`, `values()` and `items()` return **view objects**. They always reflect the latest state of the dictionary, without you calling the method again.

Example

```
d = {"a": 1, "b": 2}
k = d.keys()
print(k)
d["c"] = 3
print(k)         # updated automatically
```

Output

```
dict_keys(['a', 'b'])
dict_keys(['a', 'b', 'c'])
```

Set operations on keys

Example

```
a = {"x": 1, "y": 2, "z": 3}
b = {"y": 20, "z": 30, "w": 40}
print(a.keys() & b.keys())    # common keys
print(a.keys() | b.keys())    # all keys
print(a.keys() - b.keys())    # keys only in a
print(a.keys() ^ b.keys())    # keys in only one
```

Output

```
{'y', 'z'}
{'w', 'y', 'z', 'x'}
{'x'}
{'w', 'x'}
```

16. defaultdict and Counter (Bonus)

These come from the `collections` module and make common tasks easier.

defaultdict - no KeyError, automatic default value

Example

```
from collections import defaultdict
d = defaultdict(int)          # missing keys start at 0
for ch in "banana":
    d[ch] += 1
print(dict(d))
```

Output

```
{'b': 1, 'a': 3, 'n': 2}
```

Example

```
from collections import defaultdict
groups = defaultdict(list)    # missing keys start as empty list
for name, city in [("Riya", "Delhi"), ("Aman", "Pune"), ("Neha", "Delhi")]:
    groups[city].append(name)
print(dict(groups))
```

Output

```
{'Delhi': ['Riya', 'Neha'], 'Pune': ['Aman']}
```

Counter - count things quickly

Example

```
from collections import Counter
c = Counter("banana")
print(c)
print(c["a"])
print(c.most_common(2))
words = "to be or not to be".split()
print(Counter(words))
```

Output

```
Counter({'a': 3, 'n': 2, 'b': 1})
3
[('a', 3), ('n', 2)]
Counter({'to': 2, 'be': 2, 'or': 1, 'not': 1})
```

17. Real-Life Mini Programs

Program 1: Count character frequency

Example

```
text = "hello world"
freq = {}
for ch in text:
    freq[ch] = freq.get(ch, 0) + 1
print(freq)
```

Output

```
{'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1, 'w': 1, 'r': 1, 'd': 1}
```

Program 2: Count word frequency

Example

```
sentence = "the cat and the dog and the bird"
count = {}
for w in sentence.split():
    count[w] = count.get(w, 0) + 1
print(count)
```

Output

```
{'the': 3, 'cat': 1, 'and': 2, 'dog': 1, 'bird': 1}
```

Program 3: Student result card

Example

```
marks = {"maths": 90, "physics": 72, "chem": 48, "english": 85}
total = sum(marks.values())
percent = total / len(marks)
print("Total:", total)
print("Percentage:", percent)
failed = [s for s, m in marks.items() if m < 50]
print("Failed in:", failed)
print("Best subject:", max(marks, key=marks.get))
```

Output

```
Total: 295
Percentage: 73.75
Failed in: ['chem']
Best subject: maths
```

Program 4: Simple phone book

Example

```
book = {}
book["Riya"] = "98100"
book["Aman"] = "98200"
name = "Aman"
if name in book:
    print(name, "->", book[name])
else:
    print("Not found")
del book["Riya"]
print(book)
```

Output

```
Aman -> 98200
{'Aman': '98200'}
```

Program 5: Group words by first letter

Example

```
words = ["apple", "avocado", "banana", "blueberry", "cherry"]
groups = {}
for w in words:
    groups.setdefault(w[0], []).append(w)
print(groups)
```

Output

```
{'a': ['apple', 'avocado'], 'b': ['banana', 'blueberry'], 'c': ['cherry']}
```

Program 6: Find duplicates in a list

Example

```
nums = [1, 2, 3, 2, 4, 1, 5, 2]
seen = {}
for n in nums:
    seen[n] = seen.get(n, 0) + 1
dups = [n for n, c in seen.items() if c > 1]
print(dups)
```

Output

```
[1, 2]
```

18. Common Mistakes

Mistake 1: Using a list as a key

Example

```
try:
    d = {[1, 2]: "x"}
except TypeError as e:
    print("Error:", e)
d = {(1, 2): "x"}      # fix: use a tuple
print(d)
```

Output

```
Error: unhashable type: 'list'
{(1, 2): 'x'}
```

Mistake 2: Accessing a missing key with [ ]

Example

```
d = {"a": 1}
print(d.get("b", 0))   # fix: use get()
```

Output

```
0
```

Mistake 3: fromkeys with a list shares one list

Example

```
d = dict.fromkeys(["a", "b"], [])
d["a"].append(1)
print(d)                # both changed!
d = {k: [] for k in ["a", "b"]}     # fix: comprehension
d["a"].append(1)
print(d)
```

Output

```
{'a': [1], 'b': [1]}
{'a': [1], 'b': []}
```

Mistake 4: Changing size while looping

Example

```
d = {"a": 1, "b": 2}
try:
    for k in d:
        d["new"] = 0
except RuntimeError as e:
    print("Error:", e)
```

Output

```
Error: dictionary changed size during iteration
```

Mistake 5: Thinking "in" checks values

Example

```
d = {"a": 1}
print(1 in d)             # False, 1 is a value
print(1 in d.values())    # True
```

Output

```
False
True
```

Mistake 6: Expecting = to copy

b = a makes both names point to the same dictionary. Use `a.copy()` or `copy.deepcopy(a)` (Chapter 14).

19. Quick Reference Table

Task	Code	Result / Note
Create	d = {"a": 1}	new dictionary
Access	d["a"]	value, KeyError if missing
Safe access	d.get("a", 0)	value or default
Add / change	d["b"] = 2	adds or updates
Update many	d.update({"c": 3})	merge in place
Add if missing	d.setdefault("k", 0)	returns value
Remove key	d.pop("a")	returns value
Remove last	d.popitem()	returns (key, value)
Delete key	del d["a"]	no return
Empty it	d.clear()	becomes {}
All keys	d.keys()	view of keys
All values	d.values()	view of values
All pairs	d.items()	view of (k, v)
Copy	d.copy()	shallow copy
New from keys	dict.fromkeys(keys, v)	same value for all
Check key	"a" in d	True / False
Count pairs	len(d)	integer
Merge (3.9+)	d1 | d2	new dictionary
Sort by value	dict(sorted(d.items(), key=lambda i: i[1]))	new sorted dict
Deep copy	copy.deepcopy(d)	fully independent

20. Practice Questions with Answers

Try each question yourself first, then compare with the answer.

Q1. Create a dictionary of 3 friends and their ages, then print the age of the second friend.

Answer

```
f = {"Riya": 19, "Aman": 20, "Neha": 18}
print(f["Aman"])
```

Output

```
20
```

Q2. Add a new friend, change one age, and remove one friend.

Answer

```
f = {"Riya": 19, "Aman": 20, "Neha": 18}
f["Kabir"] = 21
f["Neha"] = 19
f.pop("Riya")
print(f)
```

Output

```
{'Aman': 20, 'Neha': 19, 'Kabir': 21}
```

Q3. Print the sum and average of all values.

Answer

```
f = {"Riya": 19, "Aman": 20, "Neha": 18}
print(sum(f.values()), sum(f.values()) / len(f))
```

Output

```
57 19.0
```

Q4. Make a dictionary of squares from 1 to 5 using comprehension.

Answer

```
print({n: n ** 2 for n in range(1, 6)})
```

Output

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Q5. Merge two dictionaries.

Answer

```
a = {"x": 1}
b = {"y": 2}
print({**a, **b})
```

Output

```
{'x': 1, 'y': 2}
```

Q6. Swap the keys and values.

Answer

```
d = {"a": 1, "b": 2}
print({v: k for k, v in d.items()})
```

Output

```
{1: 'a', 2: 'b'}
```

Q7. Find the key with the maximum value.

Answer

```
d = {"a": 5, "b": 9, "c": 2}
print(max(d, key=d.get))
```

Output

```
b
```

Q8. Count how many times each letter appears in 'mississippi'.

Answer

```
s = "mississippi"
c = {}
for ch in s:
    c[ch] = c.get(ch, 0) + 1
print(c)
```

Output

```
{'m': 1, 'i': 4, 's': 4, 'p': 2}
```

Q9. Remove all items with value less than 50.

Answer

```
m = {"A": 45, "B": 82, "C": 67, "D": 30}
m = {k: v for k, v in m.items() if v >= 50}
print(m)
```

Output

```
{'B': 82, 'C': 67}
```

Q10. Sort a dictionary by value, highest first.

Answer

```
d = {"a": 3, "b": 7, "c": 5}
print(dict(sorted(d.items(), key=lambda i: i[1], reverse=True)))
```

Output

```
{'b': 7, 'c': 5, 'a': 3}
```

Well done! You have now covered every concept and every method of the Python dictionary. Practise by typing the examples yourself and changing the values.